NOM : Matricule :

PRÉNOM : SECTION :

Compléments d'Algorithmique – Examen du 19 juin 2025

Consignes

- Commencez par indiquer votre nom, prénom, section et matricule au-dessus de chaque feuille.
- Répondez aux questions **directement sur les feuilles d'énoncés**.
- Les notes de cours et personnelles **ne sont pas autorisées**.
- Si ce n'est pas explicitement demandé dans l'énoncé :
  - **vous ne devez pas tester que les préconditions sont respectées** ;
  - **vos fonctions ne doivent pas avoir d'effet de bord** : les objets mutables passés en paramètres ne doivent pas être modifiés et rien ne doit être affiché à l'écran.

Problème 1 : Petites questions diverses

Soyez précis·e mais concis·e pour répondre aux questions suivantes.

Question 1.a. Donnez deux usages des IA génératives **autorisés** par la charte IA de l'UMONS.

Réponse

Question 1.b. Donnez deux usages des IA génératives **interdits** par la charte IA de l'UMONS.

Réponse

Question 1.c. Dans le domaine des IA génératives, expliquez ce qu'est un **corpus** ? Citez un risque lié à cette notion.

Réponse

Question 1.d. Donnez deux différences et deux points communs entre un tableau en C et une liste Python.

Réponse

Question 1.e. Comment décririez-vous les principes et particularités de la **programmation évenementielle** ? Dans quel type de code l'avez-vous utilisée ?

Réponse

Question 1.f. Dans une GUI construite avec Tkinter, à quoi sert un **Canvas** ? Donnez un exemple où un Canvas pourrait être utilisé dans le cadre d'un programme qui fait des statistiques.

Réponse

Problème 2 : Mémoïsation et mémoires de TP

Pendant les travaux pratiques, vous avez eu l'occasion de tester différents algorithmes pour résoudre le problème du voyageur de commerce (ou "travelling salesman problem" en anglais). La technique qui nous intéresse ici est celle de la récursivité avec mémoïsation.

Brièvement, nous vous rappelons que le problème consiste à trouver, pour une liste de villes donnée et des coûts de déplacement entre chaque paire de villes, le cycle le plus court passant par toutes les villes exactement une fois (et, étant un cycle, se terminant par la même ville où il a commencé).

Comme implémentée aux travaux pratiques, vous pouvez considérer qu'une classe TSP existe. Dans le cadre de ce qui est intéressant pour cet exercice, celle-ci possède une liste $_cities$ contenant des nombres de 0 à $n-1$ représentant les n villes ainsi qu'une méthode $cities$ qui renvoie cette liste. Elle possède également une méthode $cost_between(u, v)$ qui prend en paramètres deux villes u et v et renvoie le coût du trajet entre celles-ci.

A l'aide de votre mémoire sans faille, vous vous souvenez de la solution proposée pour cet exercice (ou vous la conjurez par magie si vous ne l'avez jamais regardée ...). Ca va être trop facile ...

```
1  class MemoizationSolver:
2      def __init__(self, tsp: TSP):
3          self.tsp = tsp
4          self.it = 0
5
6      def tsp_rec(self, s: int, f: int, cities: list[int]):
7          if len(cities) == 0:
8              return ([f, s], self.tsp.cost_between(f, s))
9          else:
10             min_cities = None
11             min_cost = float("+inf")
12             for i, city in enumerate(cities):
13                 perm, c = self.tsp_rec(s, city, cities[:i] + cities[i+1:])
14                 new_cities = [f] + perm
15                 new_cost = c + self.tsp.cost_between(f, city)
16                 if new_cost < min_cost:
17                     min_cities = new_cities
18                     min_cost = new_cost
19             if min_cities is not None:
20                 return (min_cities, min_cost)
21
22     def solve(self) -> tuple[list[int], int]:
23         cities = self.tsp.cities()
24         return self.tsp_rec(cities[0], cities[0], tuple(cities[1:]))[0]
```

... Ou peut-être pas si facile que ça, oups. Il semblerait que votre mémoire sans faille ait tout de même une faille ou l'autre. Vous n'avez pas mémoire de mémoïsation, et votre programme non plus.

Question 2.a. Pouvez-vous indiquer les lignes de code à ajouter à cette classe pour mémoriser les triplets $(s, f, cities)$ pour lesquels le problème a déjà été calculé ? Soyez précis·e·s en indiquant où les lignes de code viennent s'insérer.

Réponse

Imaginons maintenant une variante de notre problème où notre pauvre voyageur de commerce se fatigue au fur et à mesure de son voyage. Dans notre problème, cela se symbolise par le fait que le coût du trajet entre deux villes varie selon le nombre de villes ayant déjà été visitées. La méthode $cost_between$ est donc maintenant modifiée pour prendre un troisième paramètre, $travelled$, qui est un entier représentant le nombre de villes ayant déjà été visitées.

Question 2.b. Quels changements doivent être effectués dans MemoizationSolver pour prendre en compte cette nouvelle contrainte ? Considérez que la méthode $cost_between$ a déjà été réimplémentée pour vous tel que décrit plus haut (**Donc vous ne devez pas l'écrire, d'autant qu'il n'a pas été précisé exactement comment celle-ci est implémentée, sachez simplement qu'elle dépend de travelled**).

Réponse

Problème 3 : AlgXiv

Aujourd'hui, vous allez aider des chercheurs à organiser leurs articles scientifiques. Pour cela, vous allez concevoir **AlgXiv**, un outil permettant de gérer efficacement une collection d'articles scientifiques. Votre mission consiste à modéliser les articles ainsi qu'un recueil permettant de les stocker et de les retrouver facilement via des méthodes que vous implémenterez.

Pour cela, vous modéliserez un article via une classe `Article` définie par les attributs suivants : `titre` (chaîne de caractères), `auteur` (chaîne de caractères), `doi` (chaîne de caractères, identifiant unique de l'article) et `annee` (entier représentant l'année de publication). Le DOI (Digital Object Identifier) est un identifiant unique pour chaque article scientifique ou ressource numérique en général.

Vous implémenterez également une classe `Recueil` contenant un seul attribut `articles`. Cet attribut sera une structure de données appropriée permettant de stocker les instances de la classe `Article`.

Question 3.a. Quelle structure de données serait la plus appropriée pour stocker les articles dans la classe `Recueil`, afin de pouvoir retrouver un article via son DOI en $O(\log_2 N)$ dans le pire des cas, où N est le nombre d'articles ? Justifiez votre réponse en y incluant, si nécessaire, la ou les propriétés que la structure de données doit respecter pour garantir cette complexité.

Réponse

Question 3.b. Implémentez les méthodes d'initialisation des classes `Article` et `Recueil`.

Réponse *(Utilisez les lignes pour spécifier les indentations)*

```
class Article:
```

```
class Recueil:
```

Question 3.c. Implémentez la méthode `rechercher_par_doi(doi)` de la classe `Recueil` qui permet de chercher un article via son DOI. Cette méthode doit retourner l'article correspondant ou `None` si aucun article n'est trouvé. Votre implémentation doit avoir une complexité en $O(\log_2 N)$ dans le pire des cas, où N est le nombre d'articles dans le recueil.

Réponse *(Utilisez les lignes pour spécifier les indentations)*

Question 3.d. Justifiez que votre implémentation de la méthode `rechercher_par_doi(doi)` a une complexité en $O(\log_2 N)$ dans le pire des cas, où N est le nombre d'articles dans le recueil.

Réponse

Question 3.e. Quelle est la meilleure complexité possible de la méthode `rechercher_par_titre(titre)`, permettant de chercher un article via son titre, de la classe `Recueil` utilisant la structure de données choisie à la Q3.a ? Justifiez votre réponse.

Réponse

Problème 4 : Évaluations à large échelle

Depuis quelques années, les IAs deviennent de plus en plus présentes dans notre société, et dans vos flux de réseaux sociaux. Espérant pouvoir l'utiliser pour améliorer votre orthographe dans vos rapports, vous décidez de vous renseigner sur comment l'IA peut vous aider. Vous trouvez énormément de modèles différents qui peuvent vous aider. Face aux nombreux choix devant vous, votre légendaire indécision ne vous permet pas d'en sélectionner qu'un seul. Vous voilà donc avec plusieurs modèles, parmis lesquels vous devez maintenant en sélectionner un. Vous décidez d'utiliser la métrique de *relative recall*, qui vous permettra d'évaluer à quel point un modèle détecte beaucoup de fautes d'orthographe. Le relative recall se calcule comme le nombre de fautes détectées par un modèle, divisé par le nombre de fautes *différentes* détectées par tous les modèles.

Bien que ça semble compliqué, vous voyez une manière assez simple de faire ceci. Vous avez un *très* long fichier texte contenant des phrases, chacune sur une ligne différente. Pour chaque phrase, vous allez demander à chaque modèle de vous donner le nombre n de fautes d'orthographes. Vous incrémentez de n un compteur lié au modèle afin de maintenir combien de fautes ce modèle a détecté dans le texte complet. Ensuite, vous incrémentez un compteur global avec, pour chaque phrase, le nombre maximum d'erreurs trouvées par l'ensemble des modèles, vous permettant ainsi d'avoir une *approximation* (suffisante ici) du nombre totale de fautes différentes. En y réfléchissant un peu, vous arrivez déjà à une structure assez simple d'une classe permettant de calculer ça :

```
class AI():
    def __init__(self, name):
        # plein de choses
        self.name = name
    def n_typos(self, sentence):
        # 'sentence' est de type 'str'
        # plein de choses
        return number_of_detected_typos

class AIIterator():
    def __init__(self, ...):
        # TODO
    def __next__(self):
        # TODO
```

```
class AIEvaluator():
    def __init__(self, ias, file):
        self.ias = ias # une liste d'instance d'AI
        # file = 'str' (chemin vers un fichier texte)
        self.file = file
        # nombre de typos detectees par chaque IA
        self.typos_detected = [0] * len(ias)
        # nombre totale de typo detectees
        self.total_typos = 0
    def _read_file(self):
        # TODO
    def _compute_relative_recall(self):
        # TODO
    def __iter__(self):
        # TODO
```

Vous voilà donc avec une classe `AI` qui vous permet de calculer (de manière abstraite pour vous) le nombre de fautes dans une phrase via la méthode `n_typos`, qui vous retourne un entier représentant ce nombre de fautes. Une classe `AIEvaluator` vous permet de calculer et maintenir le nombre de fautes détectées au total et par chaque modèle. Enfin, une classe `AIIterator` vous permet d'itérer sur ces résultats afin d'obtenir le relative recall de chaque modèle. Il ne vous reste plus qu'à compléter les classes `AIIterator` et `AIEvaluator` en remplaçant les TODOs.

Pour les sous-questions ci-dessous, vous n'êtes pas obligé de recopier le code ci-dessus si cela ne vous est pas utile. Veillez cependant à rendre clair ce à quoi vous répondez! (Quelle sous-question ? Quelle méthode de quelle classe ? ...) Vous êtes libre de déclarer d'autres attributs et méthodes si cela vous aide.

Question 4.a. Commencez par compléter la méthode `_read_file` de la classe `AIEvaluator`. Cette méthode représente un *générateur*, vous retournant une à une les lignes présentes dans le fichier (dont le chemin est représenté par une chaîne de caractère `file`).

Question 4.b. Continuez ensuite afin de compléter la méthode `_compute_relative_recall` de la même classe, qui s'occupera de calculer le nombre total de fautes, ainsi que le nombre de fautes détectées par chaque modèle. Le nombre total est représenté par l'attribut `total_typos`, tandis que le nombre de fautes détectées par chaque IA est représenté par une liste d'entier `typos_detected` (où `typos_detected[i]` représente le nombre de fautes détectées par l'IA en indice `i` de la liste `ias`).

Question 4.c. Enfin, complétez la méthode `AIEvaluator.__iter__` et la classe `AIIterator` afin de pouvoir itérer sur la classe `AIEvaluator`. À chaque itération, vous devez retourner un couple (tuple de deux éléments) avec le nom de l'IA (attribut `name`) et un nombre flottant représentant le relative recall (calculé comme *nombre de fautes détectées par l'IA/nombre de fautes total*).

Réponse